OpsSchool Project (The Elephant)

Details and Requirements



OPSSCHOOL

General Details

In this assignment you will build a small prod like environment

You will deploy a highly available web application on AWS, that is publicly accessible (open to the internet).

While building the infrastructure to your application, you will need to take into consideration all the subjects learned during your training.

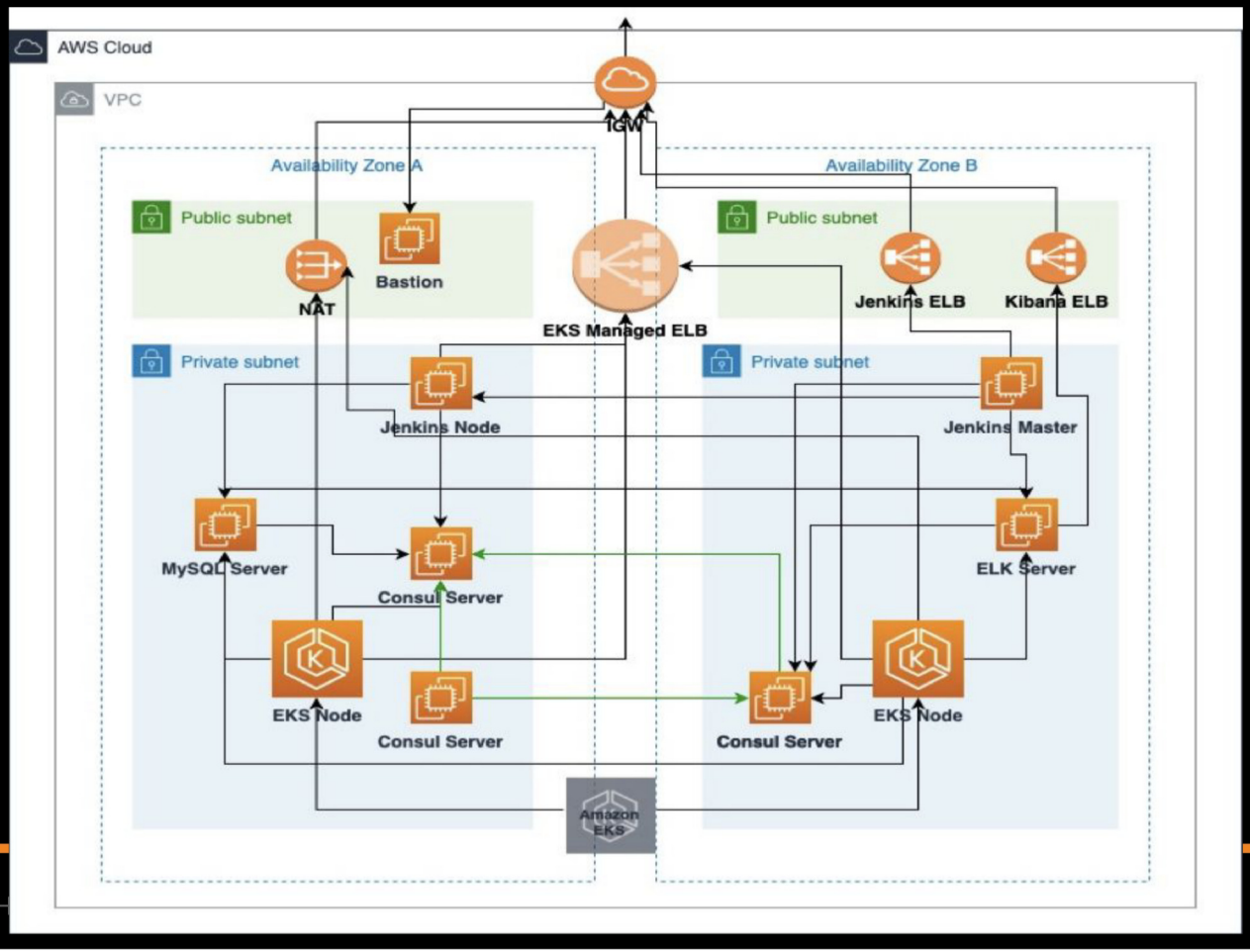
Color coding:

- White items - Required for both mid and final project presentation
- **Purple items - Required only for final project presentation**

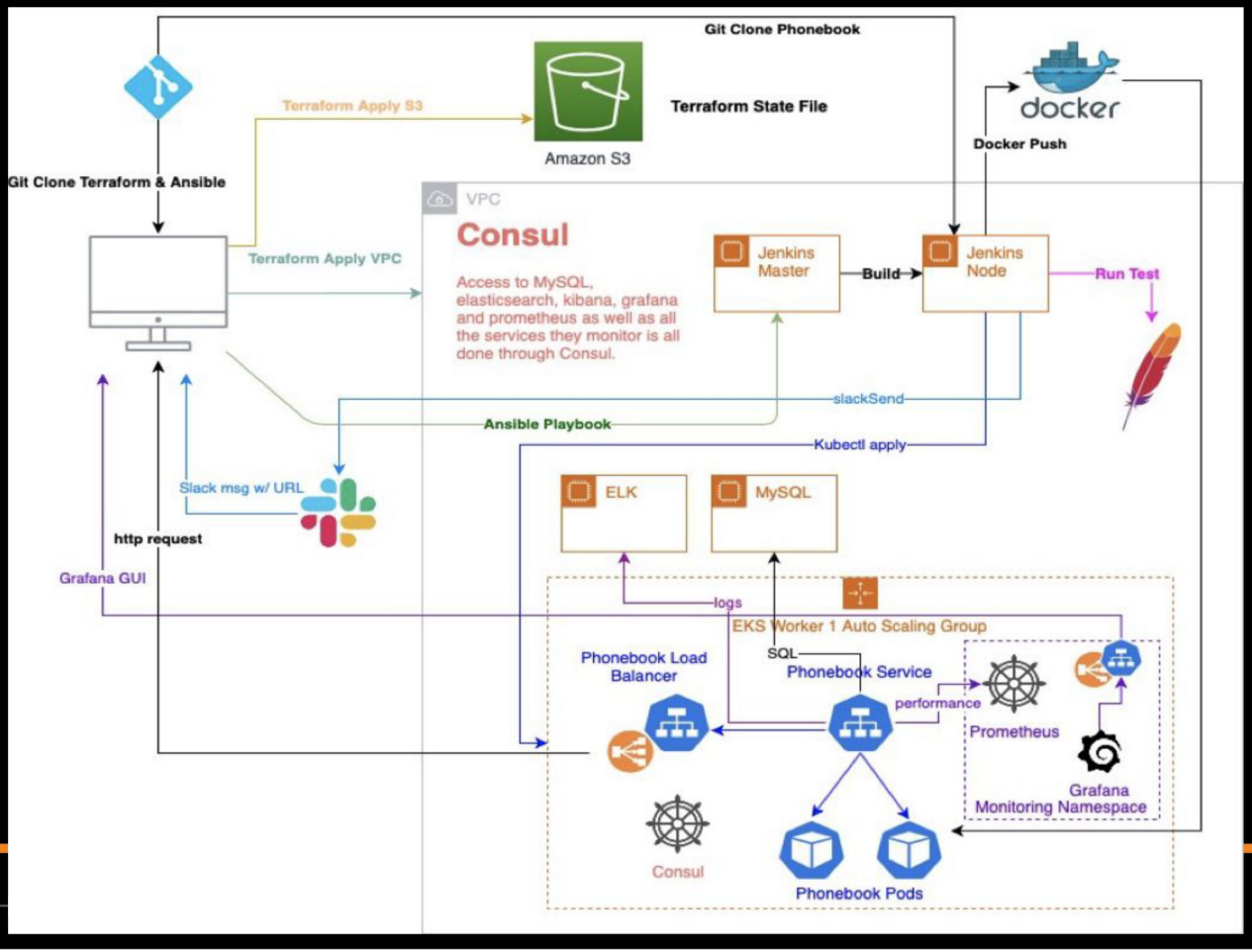
Deliverables

- Git repo (one or more) with your project
  - The repo should contain all infrastructure as code processes
  - All application code and application lifecycle processes
  - A detailed readme(s) about the project, pre-requisites, how to execute everything etc.
- A presentation that includes
  - Who am I?
  - Infrastructure architecture diagram (see example)
  - Application and CI/CD flow diagram (see example)
  - Details about the different infrastructure tools and the decisions you've made
  - Problems you encountered, and how you solved them (or tried to solve them)
  - Improvement points for the future
- Provide proof that the application and infrastructure are "up and running"

Deliverables

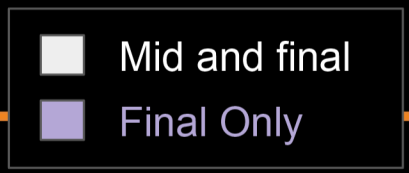


Deliverables



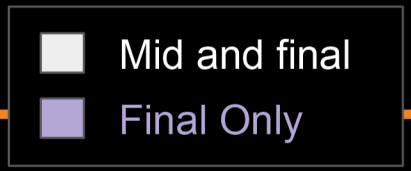
Automation

- All infrastructure resource allocation and server configuration tasks should be automated and saved as code using:
  - Terraform
  - Ansible
  - Scripts (Shell/Python/other)
- All application lifecycle tasks should be automated and saved as code using:
  - Python
  - Groovy pipelines
  - Kubernetes yaml
  - Helm
  - Ansible
  - Shell (if needed)



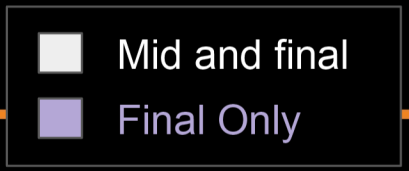
AWS

- All required VPC components for highly available infrastructure
- Public and Private subnets
  - External facing application (Kandula, Load Balancers, CI Master etc.) should be put in public subnets
  - All backend and internal systems/servers should be put in private subnets
- Separated Security Groups for each service/server
- Specific IAM roles (when needed)
- Required EC2 instances/EBS Storage and S3 storage



CI/CD Pipelines

- Create a Jenkins server and nodes (minimum 2)
- Create a CI pipeline with:
  - **SCM** - Github SCM integration for code updates (can be webhook or other)
  - **Build** - Testing for the application
  - **Packaging and Versioning** - Docker image build for application code
  - **Upload** - Pushing the artifact to dockerhub/registry
- Create CD pipeline (can be part of CI pipeline or separate):
  - Deploy artifact in kubernetes using deployment (using helm/ansible/plugins/other)



Service Discovery

- Create resilient consul infrastructure
- All infrastructure and application services are registered as consul services
  - Use appropriate health check on all your services

Mid and final

Final Only

Kandula

- Application running on K8s with resiliency
- Functionality implemented:
  - Describe instances
  - Instance Health
    - AWS
  - Stop/Start instances

Mid and final

Final Only

Kubernetes

- Create a resilient Kubernetes infrastructure
- Create a deployment of your application in kubernetes

Mid and final

Final Only

Security

- Define Granular Security Groups for each service/server
  - Limit traffic with specific server groups using Security Groups
  - Define security groups for services and not for servers

